Symbolic—Operational Separation for Lazy Probabilistic Inference in Bayesian Networks

Anonymous Draft

August 17, 2025

Abstract

We present a symbolic message-passing and lazy evaluation framework for exact inference in discrete Bayesian networks. The approach separates (i) a finite algebra of potential expressions (syntax) from (ii) their numerical realization (semantics). This enables: (a) one-time construction of a formula directed acyclic graph encoding posterior clique and marginal factors; (b) demand-driven evaluation with local normalization; (c) fine-grained cache invalidation upon evidence retraction or local distribution replacement; and (d) specialized joint probability evaluation spanning multiple cliques. We formalize the grammar, well-formedness constraints, denotational and operational semantics (including a cache transition system), and sketch soundness, completeness, normalization invariance, and locality properties.

1 Introduction

Exact inference in Bayesian networks via junction trees usually mixes two concerns: symbolic structuring (factor graph transformations) and numerical table computations. Changes in hard/soft evidence or local conditional distributions commonly force broad recomputation. We advocate a separation: a *symbolic propagation* phase constructs a reusable formula DAG; a *lazy evaluator* realizes numeric potentials only when queried. Cache entries are invalidated minimally upon structural or evidential changes.

Contributions. (1) A concise algebra of formula constructors (product, marginalization, unit, evidence indicator, node prior, reference) with normalization-preserving semantics. (2) Symbolic junction-tree style propagation producing message and posterior formulas independent of concrete evidence values. (3) Demand-driven evaluation with recursive cache invalidation bounded to dependency subgraphs. (4) A hybrid strategy for multi-clique joint queries that retracts and restores evidence only when required. (5) Formalization of syntax, semantics, and correctness properties.

2 Related Work

Lazy propagation [1] separates message structure from evaluation but typically still materializes intermediate numeric tables under new evidence. Arithmetic circuits / compiled representations (e.g. [2]) precompile global structures; our approach constructs a lightweight runtime formula DAG without global elimination ordering compilation. AND/OR search spaces, factor caching, and lifted inference are complementary techniques.

3 Syntax

3.1 Basic Sets

Let Var be a finite set of variables. Each $X \in \mathsf{Var}$ has a finite domain $\mathsf{Dom}(X) = \{x_0, \ldots, x_{k-1}\}$; define $\mathsf{Lev}(X) = \{0, \ldots, k-1\}$. For $S \subseteq \mathsf{Var}$, $\mathsf{Assign}(S) = \prod_{X \in S} \mathsf{Dom}(X)$.

Formula identifiers form a finite prefix of $\mathbb{N}$ denoted Id.

CPT Semantics. A `CPT` enumerates one row per distinct assignment to the parent variables. For a node X with parent set $P = \{Y_1, \ldots, Y_m\}$, let $R = \prod_{i=1}^{m} |\mathsf{Dom}(Y_i)|$. The CPT must contain exactly R entries; each `Condition` lists assignments whose variables cover P exactly once (order immaterial). Each corresponding `ProbVector` has length $|\mathsf{Dom}(X)|$ and sums to 1. If a `PotentialSpec` is given instead of a `CPT`, it supplies either (i) an unconditional prior (no parents) or (ii) a total function mapping $\mathsf{Assign}(\{X\} \cup P)$ to $[0, 1]$ normalized per parent assignment.

3.2 Domain Function and Size

Define $\mathrm{dom}(F)$ recursively:

$$\begin{aligned} \mathrm{dom}(\Phi(X)) &= \{X\} \cup \mathsf{Pa}(X), & \mathrm{dom}(\epsilon(X)) &= \{X\}, \\ \mathrm{dom}(\Pi(F_1, \ldots, F_n)) &= \bigcup_i \mathrm{dom}(F_i), & \mathrm{dom}(\Sigma_S(F)) &= S \cap \mathrm{dom}(F), \\ \mathrm{dom}(\mathrm{ref}(i)) &= \mathrm{dom}(F_i), & \mathrm{dom}(\mathbf{1}) &= \emptyset. \end{aligned}$$

Let $\vec{k}_F$ be the vector of $|\mathsf{Dom}(X)|$ in the order induced by $\mathrm{dom}(F)$. Define $|F| = \prod \vec{k}_F$ with $|\mathbf{1}| = 1$.

3.3 Grammar (EBNF)

We now introduce the concrete grammar used for (i) declaring a Bayesian network, (ii) stating queries and evidence, and (iii) representing the internal symbolic formulas that the system constructs. Rather than presenting one monolithic block, we proceed fragment by fragment. For each fragment we first describe in prose what problem it solves and what design choices constrain its shape; then we give the EBNF productions; finally we justify the well-formedness side conditions that (for readability) appear as comments inside the grammar. This sequencing parallels how a user (or an implementation) incrementally acquires the necessary concepts: characters and tokens first, then collections and assignments, then conditional structure, and so forth, culminating in the compiled algebra.

3.3.1 Lexical Primitives

The lexical layer introduces the smallest building blocks. An `Identifier` is the syntactic unit from which we draw both variable names and the names of their discrete levels. We intentionally do not create distinct token categories (e.g. `VarId` vs. `LevelId`); instead, the role of an identifier is determined later by the context in which it appears (a level list versus a node declaration head). This uniformity simplifies parsing and later allows lightweight transformations that operate uniformly over identifiers without tracking their categories until semantic validation. Integer literals furnish a supply of natural numbers used principally for formula identifiers (references), while real literals stand for probability masses. We restrict real literals (syntactically) to the closed unit interval so that malformed probabilities (e.g. negative numbers) can be rejected at parse time rather than after partial semantic construction.

```
Identifier ::= /* letter (letter | digit | '_' )* (non-empty) */
Var ::= Identifier
Level ::= Identifier
IntegerLiteral ::= /* non-negative integer */
RealLiteral ::= /* decimal in [0,1] inclusive */
Probability ::= RealLiteral (* intended in [0,1] *)
```

The non-emptiness requirement and restricted character class ensure that identifiers cannot be confused with delimiters or punctuation tokens, keeping later pretty-printing unambiguous. The probability annotation communicates a contract: subsequent grammar fragments may assume probabilities are already range-checked, which simplifies semantic rules for normalization.

3.3.2 Collections, Assignments, Distributions

The next fragment groups identifiers into ordered finite collections and introduces partial assignments and basic probability tables. A `LevelList` fixes the finite domain of a variable; a `VarList` collects distinct variable names (distinctness assures that the product space we form from their domains is well-defined without duplicate dimensions). `VarListOpt` allows an explicit empty parent list so that the absence of parents is syntactically visible rather than implicit. An `Assignment` binds a single variable to one of its declared levels. A `Condition` is a finite set of such bindings and will later serve as the left-hand side key of a CPT row. Finally, a `ProbVector` encodes the categorical distribution over a variable conditioned on its parents (or an unconditional prior when there are no parents). The side comment "sums to 1" flags a semantic check: the vector's entries must sum exactly to one so that the vector denotes a normalized probability distribution.

```
LevelList ::= "[" Level { "," Level } "]" (* distinct levels *)
ProbVector ::= "[" Probability { "," Probability } "]" (* sums to 1 *)
VarList ::= "[" Var { "," Var } "]" (* distinct *)
VarListOpt ::= "[" [ Var { "," Var } ] "]"
Assignment ::= Var "=" Level
Condition ::= "{" [ Assignment { "," Assignment } ] "}" (* covers parent set when in CPT *)
```

Requiring distinct elements in all lists prevents a later evaluation routine from encountering the same dimension twice (which would force ad hoc disambiguation rules). Mandating that a `Condition` appearing in a CPT (next fragment) covers each parent exactly once ensures the CPT behaves as a total function from parent assignments to distributions, eliminating the possibility of partial or conflicting entries.

3.3.3 CPTs and Potentials

This fragment provides two alternative mechanisms for supplying a node's local conditional distribution. A `CPT` explicitly lists (row by row) the conditional distribution of a child variable for every complete assignment to its parents. A `PotentialSpec` gives either an unconditional prior distribution for a parentless node or a reference to an external (possibly procedural) potential function together with the exact list of variables on which it depends. The function form makes it possible to integrate non-tabular or deferred distributions (e.g., ones computed on demand) while still letting the symbolic layer reason about domains.

```
CptEntry ::= Condition "->" ProbVector (* |ProbVector| = |Dom(child)| *)
CPT ::= "{" CptEntry { "," CptEntry } "}"
PotentialSpec ::= "prior" "(" ProbVector ")"
                                | "function" "(" VarList ")" (* external symbolic potential *)
```

Each conditional entry must (semantically) appear exactly once: if any parent assignment were missing the local conditional would be undefined for that configuration; if a parent assignment appeared twice evaluation would be ambiguous. The probability vector length check enforces

arity alignment between the declared domain of the child and its row; without this, later table multiplications could mismatch dimensions silently. For function potentials, the explicit variable list makes the domain of the resulting symbolic formula decidable without executing the function body; this is essential for static construction of product and marginal operations.

3.3.4 Nodes and Network

Here we scale from local specifications to the whole structure. A `NodeDecl` associates a variable with its domain (level list), an explicit list of parent variables (possibly empty), and an optional specification of how its conditional distribution is supplied (either a CPT or a potential). The surrounding `Network` production gathers a finite set of such declarations. The order among declarations is not semantically significant, but an implementation may exploit declaration order to perform early validation (e.g., ensuring that referenced parents have been declared).

```
NodeDecl ::= "node" Var ":" "levels" "=" LevelList
                                       "parents" "=" VarListOpt
                                       ( "cpt" "=" CPT | "potential" "=" PotentialSpec )?
Network ::= "network" "{" NodeDecl { NodeDecl } "}"
```

By centralizing level list declarations we guarantee one authoritative domain for each variable, which the semantic layer uses when allocating factor tables. Allowing a node declaration without an immediate CPT/potential supports workflows where structure learning precedes parameter estimation. A separate acyclicity check (not encoded directly in the context-free grammar) ensures the declared parent relation defines a DAG so that the global joint factorization exists.

Example 1 (Asia network CPT encoding)**.** Consider the well-known Asia network fragment with variables `Smoker` and `LungCancer`. In code (see the provided implementation), `Smoker` is a parentless binary variable with levels $[T, F]$ and prior $P(\texttt{Smoker} = T) = 0.5$. `LungCancer` has parent `Smoker` and conditional probabilities $P(\texttt{LungCancer} = T \mid \texttt{Smoker} = T) = 0.1$ and $P(\texttt{LungCancer} = T \mid \texttt{Smoker} = F) = 0.01$. Using the grammar, a minimal network fragment specifying just these two nodes can be written:

```
network {
       node Smoker : levels = [T,F]
                           parents = []
                           cpt = { {} -> [0.5,0.5] }

       node LungCancer : levels = [T,F]
                           parents = [Smoker]
                           cpt = {
                                  {Smoker=T} -> [0.1,0.9], // P(LungCancer=T|Smoker=T)=0.1
                                  {Smoker=F} -> [0.01,0.99] // P(LungCancer=T|Smoker=F)=0.01
                           }
}
```

Interpretation details:

- The level order $[T, F]$ fixes the ordering of entries in each probability vector; thus $[0.1, 0.9]$ means $P(T) = 0.1$, $P(F) = 0.9$ for `LungCancer` given the stated parent assignment.
- The empty condition (`{}`) row for `Smoker` is permissible because there are no parents; it supplies an unconditional prior.
- Each parent assignment to `Smoker` appears exactly once in the `LungCancer` CPT ensuring totality; vectors have length two matching the declared domain of `LungCancer`.
- If we were to reorder the level list (e.g. $[F, T]$) all probability vectors would have to be reordered correspondingly; keeping the level order explicit prevents ambiguity and makes parsing deterministic.

This concrete instance demonstrates how the syntactic constraints (distinct levels, exhaustive parent coverage, matching vector length) manifest in an actual specification and prepares the reader for richer multi-parent examples (e.g. `Dyspnea`) handled analogously.

3.3.5 Events and Evidence

The query fragment gives surface syntax for probabilistic questions and observations. An `Event` specifies, for a finite set of variables, a non-empty subset of admissible levels for each; probabilistically it denotes the disjunction over all full assignments consistent with those per-variable constraints. `Evidence` uses the same shape to describe observed information (hard evidence when the subset is a singleton; soft evidence when the subset contains multiple levels to be interpreted later as an indicator or likelihood vector). Designing a uniform shape for both events and evidence reduces cognitive load and lets later algorithms share inspection logic.

```
Event ::= "{" EventItem { "," EventItem } "}" (* disjoint vars *)
EventItem ::= Var ":" "[" Level { "," Level } "]" (* non-empty subset *)
Evidence ::= "{" EvidenceItem { "," EvidenceItem } "}"
EvidenceItem ::= Var ":" "[" Level { "," Level } "]"
```

Requiring that each variable occur at most once forbids contradictory constraints (e.g. giving two different level subsets for the same variable). Prohibiting empty subsets avoids implicitly encoding impossible events (probability zero) unless the user chooses to represent that explicitly by omitting the variable or by downstream mechanisms.

3.3.6 Compiled Formula Language

Finally we present the internal algebra that the symbolic construction phase generates. Users do not author these forms directly; rather, they are produced deterministically from the network structure and the current evidence. The constructors are deliberately minimal: a unit (scalar) factor; primitive node potentials; evidence indicator functions; products representing factor multiplication; marginals representing summation (projection) onto a retained set of variables; and references that allow sharing and DAG reuse. Minimality makes it feasible to prove properties such as normalization invariance and locality by structural induction, since there are few cases to consider.

```
Formula ::= Unit | NodePotential | EvidenceFunction
                                 | Product | Marginal | Reference
Unit ::= "1"
NodePotential ::= "PHI(" Var ")"
EvidenceFunction ::= "EPS(" Var ")"
Reference ::= "ref(" Fid ")"
Product ::= "PI(" Formula { "," Formula } ")" (* n >= 2 after simplification *)
Marginal ::= "SIGMA{" VarList "}(" Formula ")" (* retains listed vars *)
Fid ::= IntegerLiteral
```

Imposing (by a smart constructor) that a product with only one non-unit child is simplified to that child yields a canonical shape and reduces the branching needed in proofs and algorithms. Restricting references to already constructed formulas ensures the overall graph is acyclic, which is vital for termination of both evaluation and cache invalidation passes. Marginal nodes retain exactly the listed variables; if that list equals the child's domain the node simplifies away, maintaining a reduced normal form.

Global Well-Formedness. Collecting the constraints across fragments: (1) list distinctness (levels, variables) guarantees unambiguous correspondence between positions in multi-dimensional tables and variable names, eliminating potential double counting; (2) probability vectors summing to one ensure each local specification truly denotes a distribution, which is necessary for later

proofs that normalization is preserved under our evaluation rules; (3) CPT exhaustiveness and uniqueness ensure the local conditional is a total function, preventing undefined or conflicting probabilities; (4) function potentials declare explicit support so the domain of any composed formula can be computed without executing opaque code; (5) event and evidence variable disjointness avoids contradictory constraints and keeps probability summations straightforward; (6) canonical product arity (no unary products after simplification) shrinks the structural induction space and allows concise complexity bounds; (7) marginal nodes that would be vacuous (retain every variable) are eliminated to keep the DAG minimal; (8) reference acyclicity provides a topological evaluation order and guarantees termination of invalidation; (9) network acyclicity (checked separately) ensures the joint factorization matches Bayesian semantics. Together these properties yield a grammar whose derivations always correspond to evaluable, normalizable probability expressions with predictable locality characteristics.

3.4 Domain Function and Size

Define $\mathrm{dom}(F)$ recursively:

$$\begin{aligned} \mathrm{dom}(\Phi(X)) &= \{X\} \cup \mathsf{Pa}(X), & \mathrm{dom}(\epsilon(X)) &= \{X\}, \\ \mathrm{dom}(\Pi(F_1, \ldots, F_n)) &= \bigcup_i \mathrm{dom}(F_i), & \mathrm{dom}(\Sigma_S(F)) &= S \cap \mathrm{dom}(F), \\ \mathrm{dom}(\mathrm{ref}(i)) &= \mathrm{dom}(F_i), & \mathrm{dom}(\mathbf{1}) &= \emptyset. \end{aligned}$$

Let $\vec{k}_F$ list $|\mathsf{Dom}(X)|$ for X in $\mathrm{dom}(F)$; define $|F| = \prod \vec{k}_F$ with $|\mathbf{1}| = 1$.

3.5 Symbolic Construction

This phase traverses a junction tree (or clique graph) performing a symbolic variant of message passing. Each posterior clique potential and each inter-clique message is represented as a formula DAG node built from primitives (node potentials, evidence indicators) using products and marginalizations. Construction proceeds acyclically so that all references point to earlier (already defined) formulas. Smart constructors normalize: (i) removing $\mathbf{1}$ factors, (ii) flattening nested products, (iii) eliminating duplicate child references, (iv) collapsing marginalizations that do not reduce domain, and (v) avoiding creation of reference chains. Result: a finite DAG $\mathcal{F}$ whose leaves are primitives and whose internal nodes are products or marginals, plus evidence functions.

Discussion. The compiled layer is *derived* from the network plus evidence layer. Smart constructors (Section 3.5) enforce normalization-preserving simplifications: Unit elimination, duplicate factor pruning (idempotence), marginal collapse when domains coincide, and acyclic reference enforcement. A formal predicate WF over each syntactic category (omitted here) underpins the correctness lemmas later. **Product**: Evaluate children left-to-right; multiply and normalize to obtain p; store at id.
Similar rules for marginalization (Σ), evidence, reference, unit. Node potentials are primitive unless replaced.

Cache Invalidation. Recursive procedure clear(i): if $C(i) = \bot$ stop; set $C(i) := \bot$; for each child j with dependency edge $i \to j$, invoke clear(j). This halts since dependency graph is acyclic (smart constructors).

Lemma 1 (Locality)**.** *Changing $\Phi(X)$ or $\epsilon(X)$ clears exactly the downward closure of its dependents; unaffected formula values remain valid.*

Sketch. Edges encode structural dependence. No path from modified primitive to unaffected formula implies numeric independence. Procedure traverses precisely reachable nodes; early stop at $\bot$ prevents revisiting. □

4 Joint Query Semantics

Event $A = \{(X, S_X)\}$, $S_X \subseteq \mathsf{Dom}(X)$, probability $P(A \mid \mathcal{E})$ computed:

(a) If $A = \emptyset$: return 1.

(b) If $|A| = 1$: evaluate node marginal and sum indicated levels.

(c) If variables lie in a single clique C: evaluate $\mathrm{Post}(C)$ and aggregate over satisfying assignments.

(d) Else if $\mathcal{E} = \emptyset$: perform symbolic join propagation over involved cliques (introducing temporary join formula) and sum.

(e) Else: temporarily retract evidence, compute $P(A \cup \mathcal{E})$ and $P(\mathcal{E})$, return ratio, restore evidence and cache snapshot.

Theorem 1 (Correctness)**.** *For any well-formed network, evidence $\mathcal{E}$, and event A, the algorithm returns $P(A \mid \mathcal{E})$.*

Sketch. Cases (a)–(c) follow from equivalence of symbolic posterior clique potentials to standard junction tree potentials (normalization invariance). Multi-clique case reduces to standard join over induced subgraph; evidence-bearing case applies $P(A \mid \mathcal{E}) = P(A, \mathcal{E})/P(\mathcal{E})$. Cache restoration preserves semantics by locality. □

5 Complexity

Let w be treewidth+1, $K = \max_X |\mathsf{Dom}(X)|$. Symbolic build time $O(|\mathcal{C}| + |\mathcal{S}|)$. Evaluating a formula with domain size at most w costs $O(K^w)$. Evidence updates cost proportional to number of affected formulas (at most linear in formulas referencing updated primitives). Multi-clique joint queries introduce a single additional join proportional to induced width on queried variables.

6 Extensions

Soft evidence: replace $\epsilon(X)$ indicator with likelihood vector; semantics identical except omission of normalization until combined. Continuous nodes: treat θ_X as density; sums become integrals; unit and structural algebra unchanged. Structure learning: incremental updates mutate only changed cliques, preserving formula ids for unchanged portions to maximize cache retention.

7 Conclusion

We formalized a separation between symbolic structure and numerical semantics for Bayesian network inference, producing a reusable normalized formula DAG and a lazy cache-evaluated semantics with principled invalidation. This offers a foundation for further optimizations (partial evaluation, factor sharing, abstraction) while retaining exactness guarantees.

A Additional Lemmas

Lemma 2 (Idempotent Evidence)**.** *Including $\epsilon(X)$ multiple times in a product leaves the resultant factor unchanged after normalization.*

Proof. Indicator squared equals indicator; normalization removes multiplicative constants. □

Lemma 3 (Reference Elision)**.** $\mathrm{ref}(\mathrm{ref}(i)) \equiv \mathrm{ref}(i)$ *and* $\mathrm{ref}(\mathbf{1}) \equiv \mathbf{1}$ *under* $[\![\cdot]\!]$.

Proof. Immediate by definition of reference expansion and unit semantics. □

Lemma 4 (Marginal-Product Commutation)**.** *If* $\mathrm{dom}(G) \cap (\mathrm{dom}(F) \setminus S) = \emptyset$*, then* $\Sigma_S(F \amalg G) = (\Sigma_S F) \amalg G$ *(up to normalization).*

Proof. Summation over variables disjoint from G's domain factors through multiplication; both sides normalize to same conditional distribution. □

B Notation Summary

$\Phi(X)$ Node prior / conditional potential.

$\epsilon(X)$ Evidence indicator formula for X.

$\amalg(\cdot)$ Normalized product of factors.

Σ_S Normalized marginalization onto S.

$\mathrm{ref}(i)$ Reference to formula id i.

$\mathbf{1}$ Unit (scalar) potential.

$\Psi(C, D)$ Message formula from clique C to D.

Acknowledgements

This draft accompanies an implementation emphasizing separation of syntax and semantics and lazy cache evaluation.

References

[1] A. L. Madsen and F. V. Jensen. *Lazy propagation: A junction tree inference algorithm based on lazy evaluation.* Artif. Intell. 1999.

[2] A. Darwiche. *A Differential Approach to Inference in Bayesian Networks.* J. ACM 2003.